FINANCIAL TRANSACTIONS

DATA ANALYSIS PROJECT

❖ *DATA Overview*

- ❑3 main tables: Users, Cards, Transactions
- ❑2 JSON files: train_fraud_labels, mcc_codes

Our dataset came from multiple sources.We had three main tables: Users, Cards, and Transactions.In addition, we had two JSON files:train_fraud_labels: contained fraud labels for transactionsmcc_codes: provided category descriptions for merchant codesBringing these together was the foundation of the project.

❖ *DATA CLEANING*

- ❑ ***Handled missing values and duplicates***
- ❑ ***Standardized column names and types***
- ❑ ***The first step was to clean each dataset in Python.We handled null values, removed duplicates, and fixed data types (Converting dates and numeric columns).***

❖ *Handling null values*

- **In the Users table:-**

✓**Converted num_credit_cards to numeric → filled nulls with 0 → rounded → absolute integer.**

✓**Created new column debt_to_income = num_total_debt / num_yearly_income.**

✓**Replaced infinities with NaN → filled with 0.**

✓**For the users table, the focus was on cleaning numeric columns.We fixed num_credit_cards by ensuring all values were numeric and filling missing ones with zero.We engineered a new feature debt_to_income.**

❖ Handling null values

- **In the Transactions table:-**

✓errors: nulls replaced with "No Error".

✓merchant_state: filled missing values using the most frequent state per city, fallback → "Unknown".

✓zip: filled missing values using the most frequent zip per city, fallback → "Unknown".

✓is_fraud: after merging labels, nulls filled with 0.

✓The transactions table had more missing data than the others.We replaced missing error values with “No Error.”For merchant location columns, we used a conditional strategy: filling with the most common state or zip per city, and using “Unknown” if nothing was available.Finally, once we merged the fraud labels, missing fraud flags were assumed to be not fraud and filled with zero.

❖ Data integration

- ✓Merged transactions with train_fraud_labels.json using transaction_id.
- ✓Mapped mcc_codes.json into transactions using the mcc column.
- ✓Built relations:
- ✓user_id → users ↔ transactions
- ✓card_id → cards ↔ transactions
- ✓After cleaning, we joined the different datasets.The fraud labels JSON was merged into transactions by transaction_id.The mcc_codes JSON was joined by the merchant category code.Finally, we linked users, cards, and transactions using their IDs to build a relational model.

❖ *Uploadig to SQL*

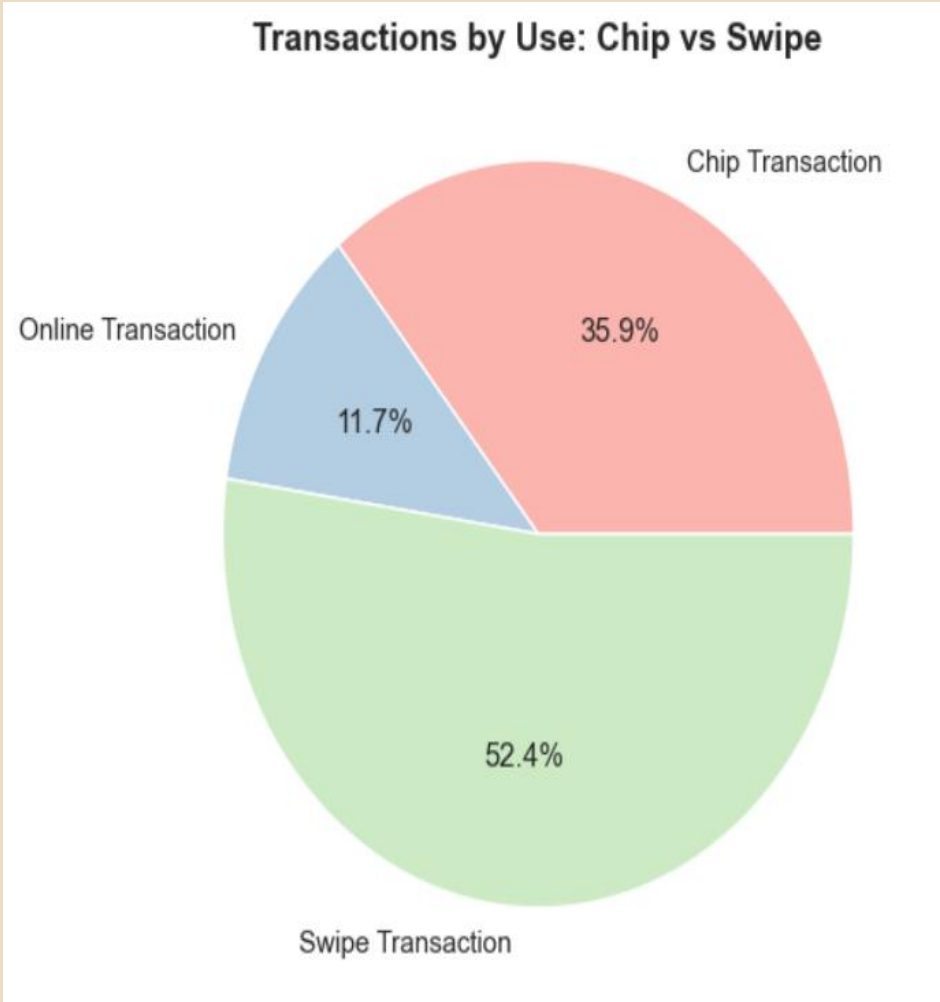
- ✓**Used SQLAlchemy to connect Python with SQL Server.**
- ✓**Uploaded cleaned data into structured SQL tables: users_data_clean , cards_data_clean , transactions_data.**
- ✓**Faced difficulty uploading the Transactions table due to its large size.**
- ✓**Solution:**

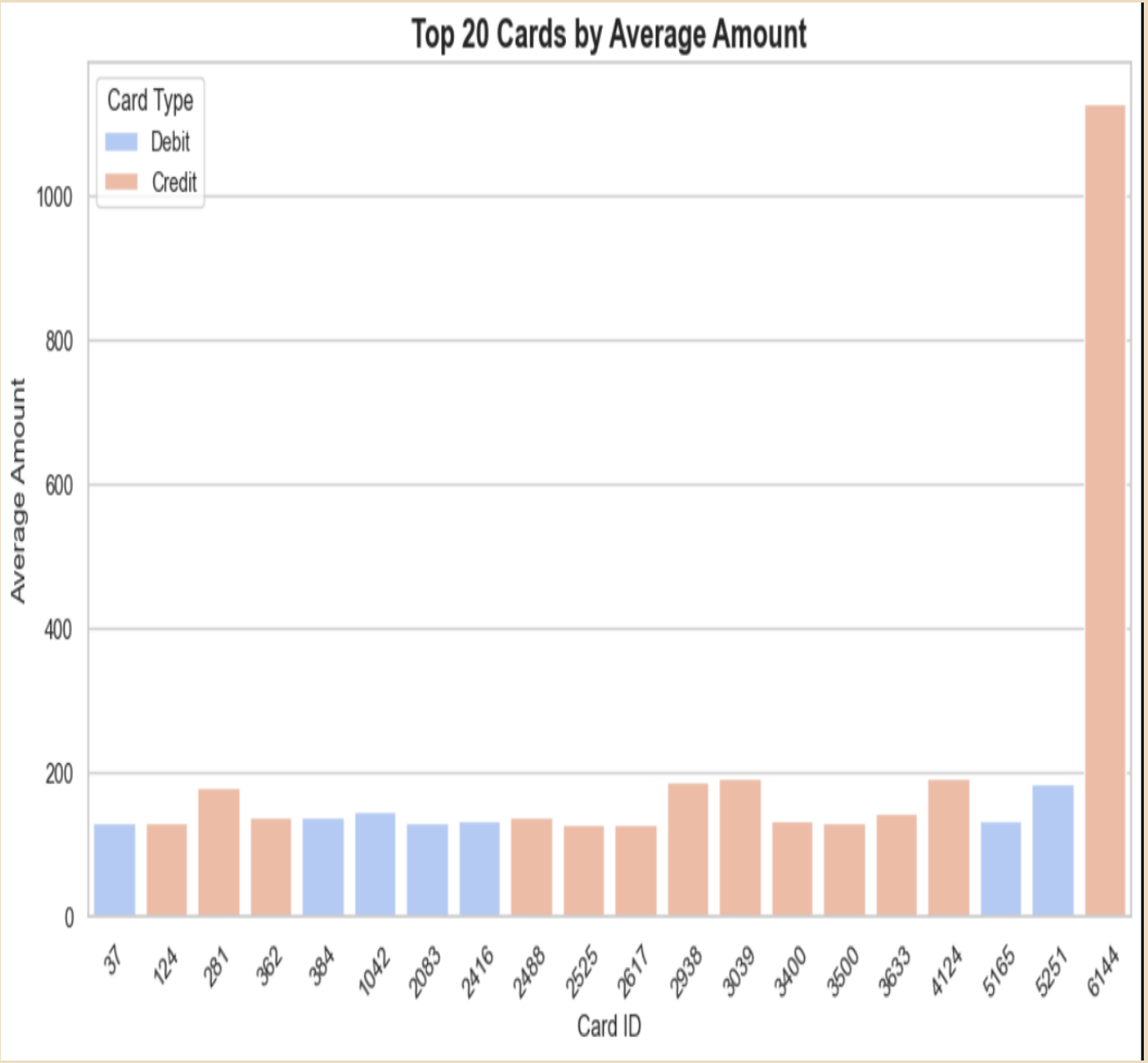
  split the Transactions table into 6 parts and uploaded each part separately.
- ✓**When we uploaded the datasets into SQL, most tables were straightforward.But the transactions table was very large, and uploading it directly caused performance and memory issues.To solve this, we divided the table into six smaller chunks and uploaded them one by one.This allowed us to successfully move all data into SQL Server.**

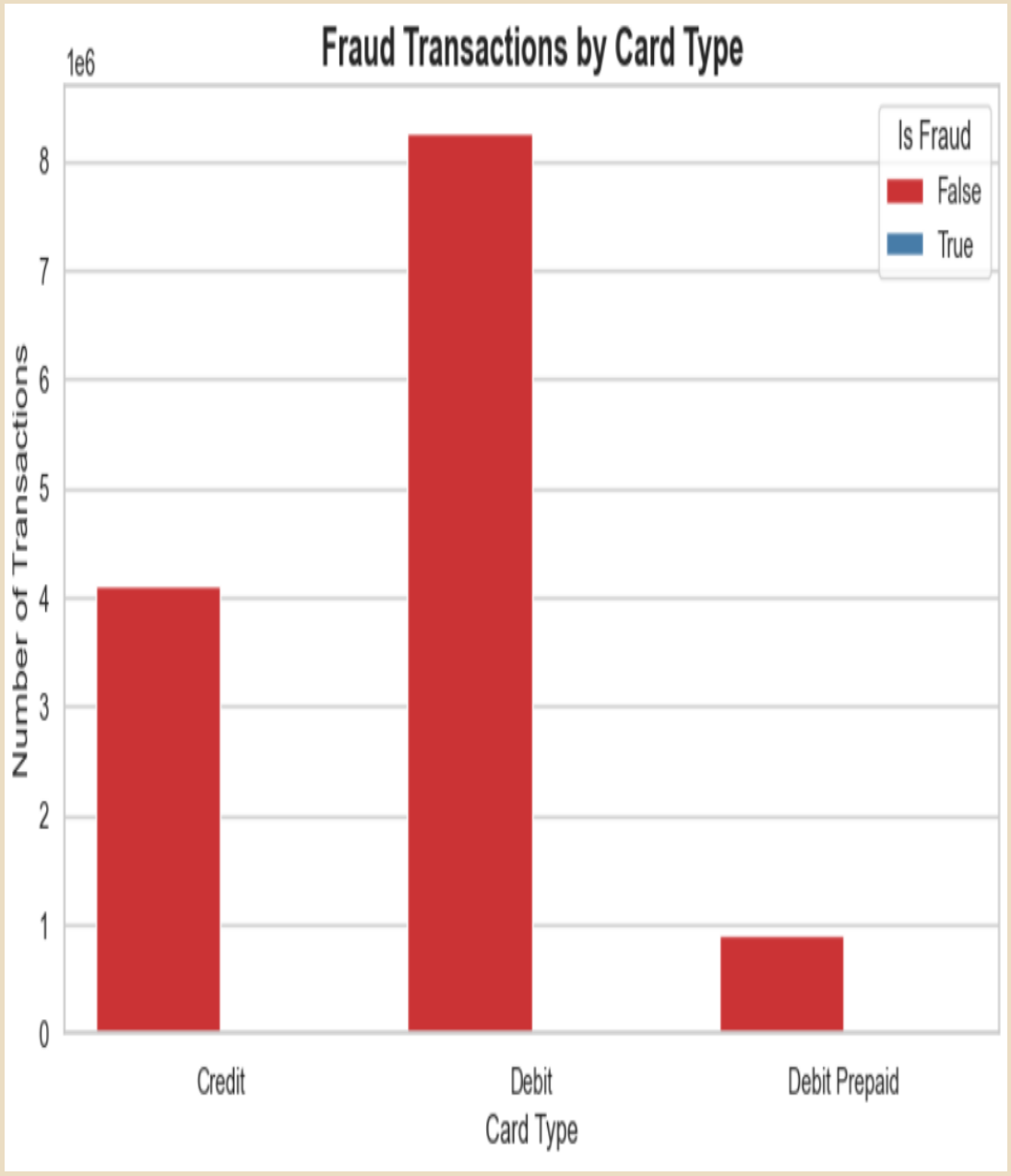
```python
from tqdm import tqdm

server = 'DESKTOP-I5RUV0T\sqlexpress'
database = 'Financial_Transactions2'

# لو Windows Authentication
connection_string = f"mssql+pyodbc://@{server}/{database}?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes"

engine = create_engine(connection_string, fast_executemany=True)



files = [
    r"C:\Users\C-LAB\Downloads\Training\Project 1\transactions_data_clean1.csv",
    r"C:\Users\C-LAB\Downloads\Training\Project 1\transactions_data_clean2.csv",
    r"C:\Users\C-LAB\Downloads\Training\Project 1\transactions_data_clean3.csv",
    r"C:\Users\C-LAB\Downloads\Training\Project 1\transactions_data_clean4.csv",
    r"C:\Users\C-LAB\Downloads\Training\Project 1\transactions_data_clean5.csv",
    r"C:\Users\C-LAB\Downloads\Training\Project 1\transactions_data_clean6.csv",
]


#  رفع البيانات chunk by chunk
chunksize = 100000
total_inserted = 0

for file in files:
    print(f"\n⏳ Uploading {file} ...")
    for chunk in tqdm(pd.read_csv(file, chunksize=chunksize, low_memory=False)):
        chunk.to_sql("transactions_data", engine, if_exists="append", index=False)
        total_inserted += len(chunk)

print(f"\n✅ Done! Uploaded {total_inserted:,} rows in total.")
```

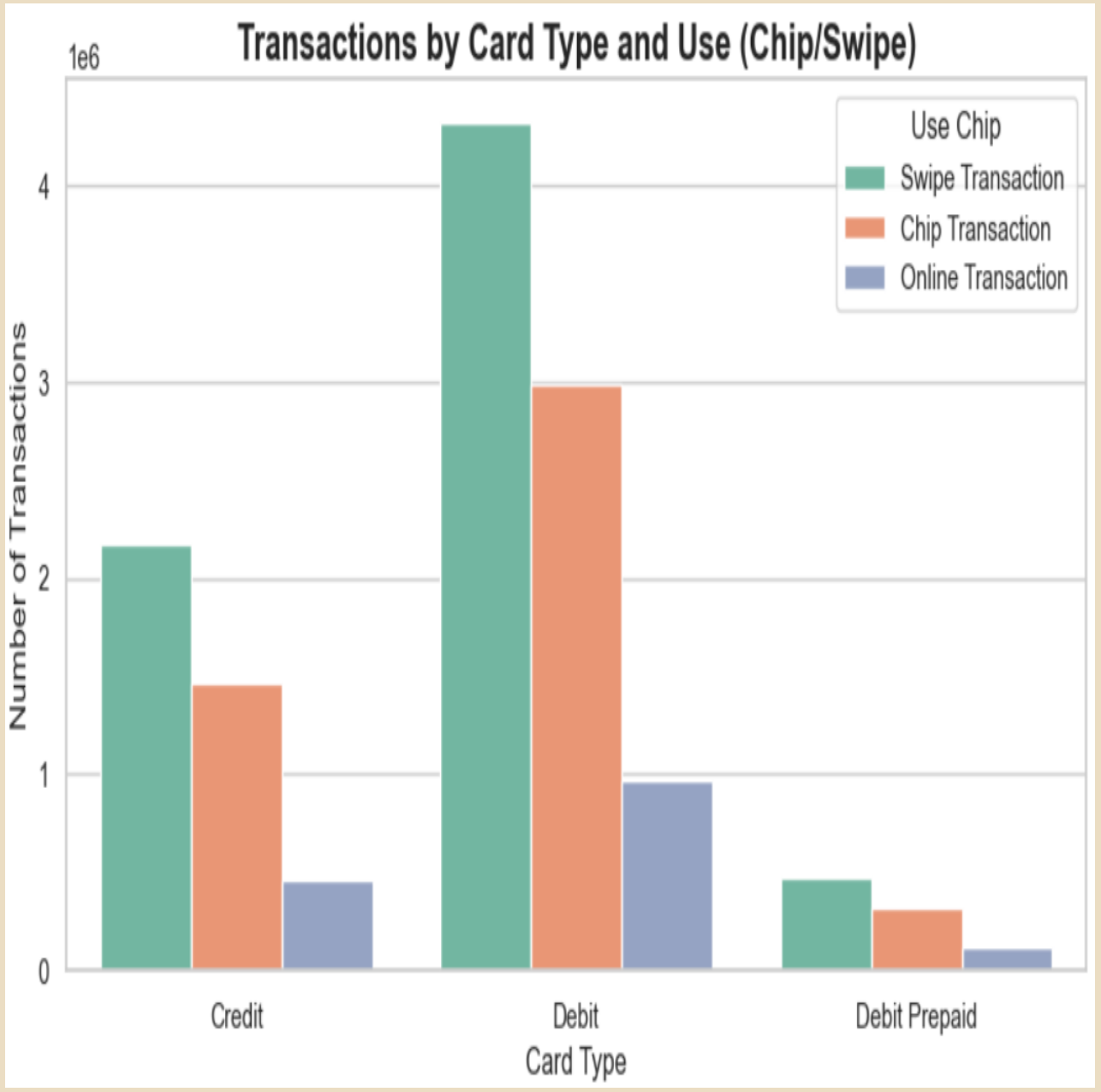
❖ *Visualization*

- **Pulled SQL tables back into Jupyter.**
- **Used Matplotlib / Seaborn for exploratory visualization.**
- **Focused on fraud patterns and transaction insights.**


Transactions by Use: Chip vs Swipe
Chip Transaction
Online Transaction
35.9%
11.7%
52.4%
Swipe Transaction


Top 20 Cards by Average Amount
Card Type
Debit
Credit
Average Amount
0
200
400
600
800
1000
37
124
281
362
384
1042
2083
2416
2488
2525
2617
2938
3039
3400
3500
3633
4124
5165
5251
6144
Card ID


Fraud Transactions by Card Type
1e6
Number of Transactions
Card Type
Credit
Debit
Debit Prepaid
Is Fraud
False
True
0
1
2
3
4
5
6
7
8


Transactions by Card Type and Use (Chip/Swipe)
1e6
Number of Transactions
Card Type
Credit
Debit
Debit Prepaid
Use Chip
Swipe Transaction
Chip Transaction
Online Transaction
0
1
2
3
4

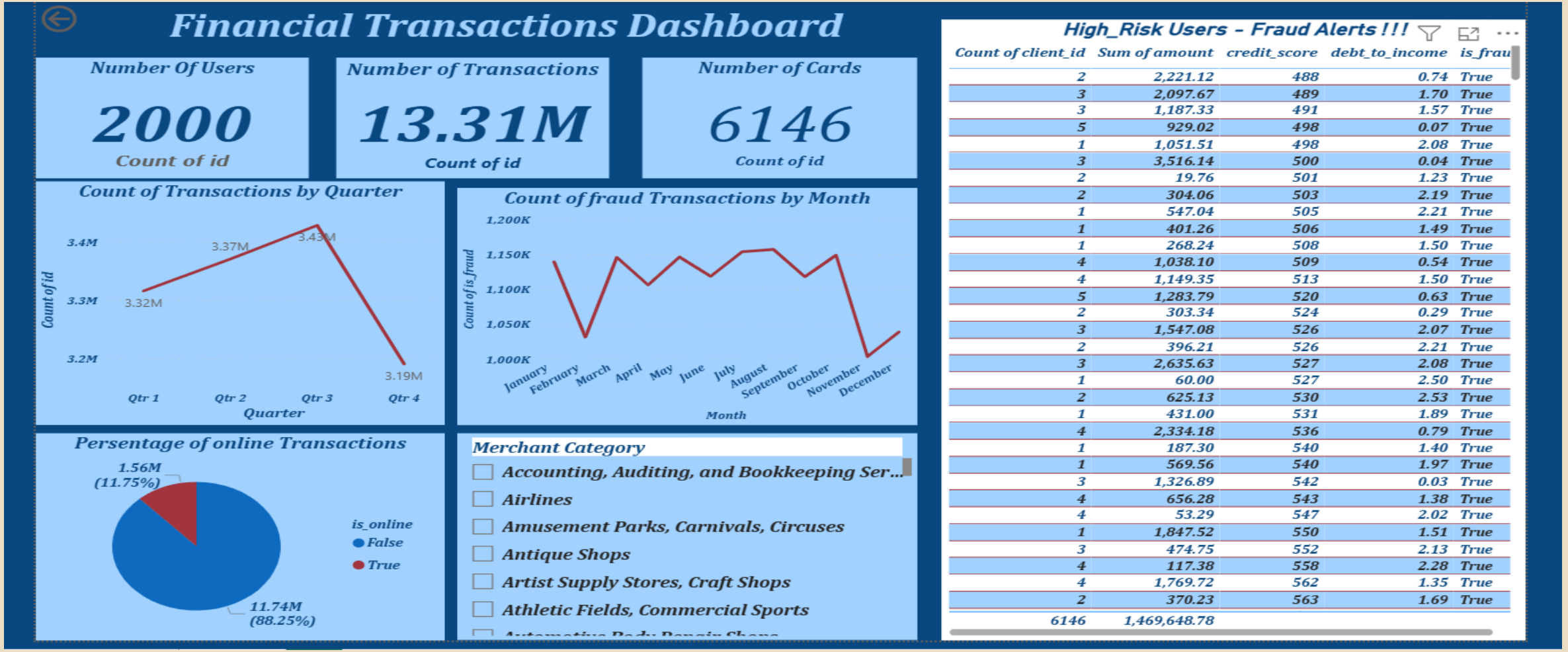
❖ *Exporting Data*

- ✓Saved final users, cards, transactions tables as CSV.
- ✓Used mainly for verification and as a backup.
- ✓We exported the cleaned DataFrames into CSV format.
- ✓This ensured we had a portable version of the data, though the main source for Power BI was SQL.

❖ *Power BI Connection*

- Connected Power BI directly to SQL Server (not CSV).
- Modeled relationships inside Power BI.
- Built dashboard with KPIs and fraud detection visuals.
- Finally, we connected Power BI directly to the SQL database.This approach was more robust than using Excel or CSV because the dashboard could directly query the structured data.The result was a live, interactive dashboard highlighting fraud detection metrics.

❖ Power BI Dashboard



High_Risk Users - Fraud Alerts !!!

Count of client_id	Sum of amount	credit_score	debt_to_income	is_frau
2	2,221.12	488	0.74	True
3	2,097.67	489	1.70	True
3	1,187.33	491	1.57	True
5	929.02	498	0.07	True
1	1,051.51	498	2.08	True
3	3,516.14	500	0.04	True
2	19.76	501	1.23	True
2	304.06	503	2.19	True
1	547.04	505	2.21	True
1	401.26	506	1.49	True
1	268.24	508	1.50	True
4	1,038.10	509	0.54	True
4	1,149.35	513	1.50	True
5	1,283.79	520	0.63	True
2	303.34	524	0.29	True
3	1,547.08	526	2.07	True
2	396.21	526	2.21	True
3	2,635.63	527	2.08	True
1	60.00	527	2.50	True
2	625.13	530	2.53	True
1	431.00	531	1.89	True
4	2,334.18	536	0.79	True
1	187.30	540	1.40	True
1	569.56	540	1.97	True
3	1,326.89	542	0.03	True
4	656.28	543	1.38	True
4	53.29	547	2.02	True
1	1,847.52	550	1.51	True
3	474.75	552	2.13	True
4	117.38	558	2.28	True
4	1,769.72	562	1.35	True
2	370.23	563	1.69	True
6146	1,469,648.78			

❖ *Recommendations*

- ✓**Handle missing values carefully: Continue improving how nulls are treated (e.g., using better imputation methods).**
- ✓**Optimize large tables: Since the transactions table is very big, keep it partitioned or indexed to improve performance.**